

PARTE 1 · FUNDAMENTOS ABSOLUTOS

De cero absoluto a nivel experto



BASES DE DATOS

MANUAL COMPLETO · SQL · POSTGRESQL · VPS · NUBE

Diseña · Crea · Consulta · Protege · Conecta · Despliega · Mantén · Escala

Contenido de esta parte

Antes de empezar

1.1 Dato frente a información

1.2 Por qué existen las bases de datos

1.3 Qué es una base de datos y qué es un gestor

1.4 Anatomía de una tabla

1.5 Tipos de datos

1.6 Valores, nulos e identificadores

1.7 Claves: primaria y extranjera

1.8 Índices

1.9 Consultas

1.10 El entorno: cliente, servidor, API, VPS y nube

Resumen de la Parte 1

Glosario de la Parte 1

Mini examen de la Parte 1

Parte 1. Fundamentos absolutos

Antes de empezar

Esta parte no necesita ningún conocimiento previo. No hace falta saber programar, ni haber usado nunca una base de datos, ni conocer ninguna palabra técnica. Todo lo que aparezca se explica la primera vez que se usa.

Al terminarla serás capaz de:

- explicar con tus palabras qué es un dato, una base de datos y un gestor de bases de datos;
- leer una tabla y decir qué es cada una de sus partes (tabla, fila, columna, registro, campo y valor);
- elegir el tipo de dato adecuado para cada columna;
- entender qué es una clave primaria, una clave extranjera, un índice y una consulta;
- seguir el recorrido de un dato desde la pantalla de una aplicación hasta el disco donde se guarda.

Cómo se organiza cada punto

Cada punto del manual sigue los mismos pasos, siempre en el mismo orden. Así sabrás dónde buscar lo que necesitas:

1. **¿Qué es?** La definición con palabras sencillas.
2. **¿Para qué sirve?** Qué problema resuelve.
3. **¿Por qué lo necesito?** Qué pasa si no lo conoces.
4. **¿Cómo funciona?** Lo que ocurre por dentro, paso a paso.
5. **Ejemplo sencillo.** El caso más pequeño posible.
6. **Ejemplo real.** Cómo aparece en un proyecto de verdad.
7. **Resultado visual.** Una imagen de lo que se ha construido, con su explicación.
8. **Ejercicio.** Algo que hacer tú mismo.
9. **Solución.** La respuesta, con su razonamiento.
10. **Error habitual.** Lo que casi todo el mundo se equivoca al principio.
11. **Buena práctica.** El hábito que usan los profesionales.
12. **Comprobación.** Cómo saber que lo has entendido.

Sobre las imágenes de esta parte. Todavía no usamos ninguna herramienta, así que las figuras son ilustraciones hechas para explicar la idea. A partir de la Parte 5 las imágenes serán capturas de resultados reales. Las pequeñas muestras de SQL que aparecen aquí son solo para ver cómo lucen: las explicaremos línea a línea en la Parte 5, así que no hace falta entenderlas todavía.

Los datos de ejemplo de esta parte

Para que todo encaje, usaremos siempre la misma pequeña tienda online. Tiene clientes y pedidos. Los clientes son Ana García, Luis Pérez y Marta Ruiz. Ana hizo dos pedidos, Marta uno y Luis ninguno. Volveremos a ellos en cada punto.

1.1 Dato frente a información

¿Qué es?

Un **dato** es un valor suelto, sin contexto. Por ejemplo: «Ana», «34», «Madrid», «45.90», «2026-03-02». Si ves uno de ellos escrito en un papel, no sabes qué significa.

La **información** es lo que obtienes cuando los datos se colocan en un contexto que les da sentido. «Ana tiene 34 años y vive en Madrid» es información: ya sabes de quién se habla, qué es el 34 y qué es Madrid.

La diferencia es la misma que entre ladrillos y una pared. Los ladrillos (datos) no sirven de nada amontonados; colocados con orden y con un plano (el contexto) forman una pared (información).

¿Para qué sirve distinguirlos?

Sirve para decidir qué guardar y cómo guardarlo. Una base de datos no guarda «información» como tal: guarda datos con la estructura suficiente para que, al pedirlos, devuelvan información. Esa estructura es lo que veremos en todo el manual.

¿Por qué lo necesito?

Imagina que alguien te da una lista con «38», «38», «37», «39». ¿Son edades? ¿Euros? ¿Grados de fiebre? ¿Tallas de zapato? Sin contexto, la lista no sirve. Si una aplicación guarda ese «38» sin decir qué significa, dentro de unos meses nadie sabrá interpretarlo, ni siquiera quien lo programó.

Cada vez que diseñes una base de datos te harás las mismas preguntas sobre cada dato: ¿de qué cosa es? ¿a quién pertenece? ¿en qué unidad está? ¿cuándo se midió?

¿Cómo funciona?

El contexto de un dato se compone de varias pistas que se combinan:

- **El nombre de la columna:** «edad», «precio» o «ciudad» dicen qué es el dato.
- **La fila a la que pertenece:** dice a quién o a qué se refiere (Ana, el pedido 101).
- **El tipo de dato:** dice si es un número, un texto o una fecha (lo veremos en el punto 1.5).
- **La unidad,** cuando hace falta: euros, kilos, años. A menudo se pone en el nombre de la columna (`precio_eur`).

Cuando las cuatro pistas están, el dato se convierte en información.

Ejemplo sencillo

Tenemos tres datos sueltos: «Ana», «34» y «Madrid». Para convertirlos en información seguimos estos pasos:

1. Decidimos de qué cosa hablamos: una persona.
2. Creamos una columna para cada característica: `nombre` , `edad` y `ciudad` .
3. Colocamos cada dato bajo su columna, en una misma fila.

4. Leemos la fila: «la persona de nombre Ana tiene 34 años y vive en Madrid».

Ejemplo real

Una tienda online guarda «101», «2026-03-02» y «45.90». Sin estructura son tres números sin sentido. Con las columnas `id_pedido`, `fecha` y `total_eur`, el sistema sabe que el pedido 101 se hizo el 2 de marzo de 2026 y costó 45,90 euros. Con eso puede mostrar el pedido al cliente, sumar las ventas del mes o enviar la factura.

Resultado visual



Figura 1.1-a. De datos sueltos a información.

Qué ves en la imagen. A la izquierda hay tres datos sueltos en cajas: no se sabe qué son. A la derecha, esos mismos datos están colocados bajo las columnas `nombre`, `edad` y `ciudad`, y ya se puede leer la frase «Ana tiene 34 años y vive en Madrid». No ha cambiado ningún dato: solo ha cambiado el contexto.

Ejercicio

1. Tienes estos datos sueltos: «Luis», «612345678» y «luis@ejemplo.com». Escribe una frase de información con ellos y di qué columna pondrías a cada dato.
2. Te dan el dato «19.95». Escribe tres significados posibles distintos y di qué información adicional necesitarías para saber cuál es el correcto.
3. Una columna se llama `dato1`. ¿Por qué es un mal nombre y cómo lo mejorarías si guarda el precio de un producto en euros?

Solución

1. «Luis puede ser contactado en el teléfono 612345678 o en el correo luis@ejemplo.com». Columnas: `nombre`, `telefono` y `email`. Cada dato responde a una pregunta distinta (quién, por dónde llamarle, por dónde escribirle).
2. Podría ser un precio en euros, una altura en metros... o unos grados de temperatura. Necesitas el nombre de la columna (`precio`, `altura`, `temperatura`), la fila a la que pertenece y la unidad.
3. `dato1` no dice qué guarda; en seis meses nadie lo recordará. Un buen nombre es `precio_eur`, porque indica qué es y en qué unidad está.

Error habitual

Guardar datos con nombres que no explican nada (`dato1` , `campo_x` , `valor`) o mezclar unidades en la misma columna (precios unos en euros y otros en dólares). Lo que hoy parece evidente mañana es un misterio.

Buena práctica

Dale a cada columna un nombre que explique su significado y, si hace falta, su unidad:

`fecha_nacimiento` , `precio_total_eur` , `peso_kg` . Escribe el nombre pensando en alguien que no conoce tu proyecto.

Comprobación

Elige cualquier dato de una aplicación que uses y responde: ¿de qué cosa es? ¿a quién pertenece? ¿en qué unidad está? Si no puedes responder a las tres preguntas, ese dato todavía no es información.

1.2 Por qué existen las bases de datos

¿Qué es?

Una base de datos es la solución a un problema muy antiguo: guardar muchos datos sin que se desordenen, se repitan, se pierdan o se contradigan. Antes de que existieran, la gente usaba cuadernos, fichas de papel, archivos sueltos en el ordenador y, más tarde, hojas de cálculo.

¿Para qué sirve?

Sirve para cuatro cosas principales:

- **Guardar** datos de forma fiable, de modo que no se pierdan si se apaga el ordenador.
- **Encontrar** un dato concreto entre millones, en una fracción de segundo.
- **Evitar duplicados y contradicciones**, guardando cada dato una sola vez.
- **Permitir el uso simultáneo**: muchas personas y programas pueden leer y modificar los datos a la vez sin pisarse unos a otros.

¿Por qué lo necesito?

Una hoja de cálculo funciona bien con pocos datos y una sola persona. Cuando el proyecto crece (miles de clientes, varios empleados, una web y una app que leen y escriben a la vez), la hoja de cálculo empieza a fallar:

- Dos personas editan el mismo archivo y una pisa los cambios de la otra.
- El mismo cliente está escrito en varias filas, y cada una dice una cosa distinta.
- Nada impide escribir «hola» en una celda de precio.
- Cualquiera que abra el archivo ve todos los datos; no se puede dar permiso solo para una parte.
- Cuando el archivo es muy grande, abrirlo y buscar en él se vuelve lentísimo.

Una base de datos está diseñada para resolver precisamente estos problemas.

¿Cómo funciona?

Una base de datos impone reglas que la hoja de cálculo no impone:

1. **Cada dato tiene un tipo.** Una columna de precios solo admite números.
2. **Cada fila se identifica de forma única.** No pueden existir dos filas indistinguibles.
3. **Los datos relacionados se guardan una vez y se enlazan.** El pedido no copia los datos del cliente: apunta a él.
4. **Hay control de acceso.** Cada usuario o programa puede tener permiso solo para lo que necesita.
5. **Los cambios son seguros.** Si algo falla a mitad de una operación, se deshace entera en lugar de dejar los datos a medias.

Ejemplo sencillo

En una hoja de cálculo de pedidos, el cliente «Ana García» aparece en tres filas, y en una de ellas el teléfono está mal escrito. ¿Cuál es el teléfono correcto de Ana? La hoja no puede decírtelo: tiene tres versiones y ninguna regla para elegir.

Ejemplo real

Una tienda tiene 20.000 pedidos de 3.000 clientes. En una base de datos, los datos de Ana se guardan una sola vez en la tabla de clientes, y sus pedidos apuntan a ella. Si cambia de teléfono, se corrige en un único sitio y todos sus pedidos quedan al día automáticamente.

Resultado visual



	A · Cliente	B · Teléfono	C · Pedido	D · Total
1	Ana García	600111222	101	45,90
2	Ana García	600111222	103	80,50
3	Ana Garcia	600111223	110	20,00

☐ Teléfono distinto: ¿cuál es el correcto?
 ☒ Nombre escrito de otra forma

Figura 1.2-a. El mismo cliente repetido en una hoja de cálculo, con datos que se contradicen.

Qué ves en la imagen. Es una hoja de cálculo de pedidos. «Ana García» aparece en tres filas. Dos filas dicen que su teléfono es 600111222, pero la tercera dice 600111223, resaltado en rojo. Además, en una fila el nombre está escrito «Ana Garcia» sin tilde. La hoja no avisa de nada: tú tienes que darte cuenta.

Ejercicio

1. Observa la figura 1.2-a y enumera dos problemas que ves.
2. Una hoja de cálculo la usan cinco personas a la vez desde sus ordenadores. Explica qué puede ocurrir si dos de ellas modifican el mismo cliente en el mismo momento.
3. Piensa en una lista que gestionas con una hoja de cálculo (gastos, clientes, tareas). Escribe una cosa que te gustaría poder impedir (por ejemplo, que se escriba una fecha imposible).

Solución

1. (a) El teléfono de Ana tiene dos valores distintos, así que no se sabe cuál vale. (b) El nombre de Ana está repetido en cada fila, y en una está escrito distinto; si cambia hay que corregirlo en todas y es fácil olvidar alguna.
2. Una de las dos podría guardar sus cambios encima de los de la otra, y una modificación se perdería sin que nadie lo note. Una base de datos coordina los accesos simultáneos para que

esto no ocurra.

3. Respuesta libre. Un ejemplo: que no se pueda escribir texto en una columna de importes. Las bases de datos lo permiten con tipos de datos y restricciones (puntos 1.5 y 3.5).

Error habitual

Pensar que una base de datos es «una hoja de cálculo más grande». Su valor no está en el tamaño, sino en las reglas (tipos, claves, permisos, transacciones) y en las relaciones entre tablas.

Buena práctica

Guarda cada dato una sola vez y enlázalo desde donde se necesite. Cuando veas el mismo dato escrito en varias filas, pregúntate si debería vivir en una tabla propia.

Comprobación

Sin mirar el texto, nombra tres problemas de una hoja de cálculo que una base de datos resuelve. Si puedes nombrar tres y explicar cómo los resuelve, has entendido para qué existen.

1.3 Qué es una base de datos y qué es un gestor

¿Qué es?

Una **base de datos** es un conjunto organizado de datos relacionados entre sí, guardados de manera que se puedan consultar, modificar y proteger.

Un **sistema gestor de bases de datos** (abreviado **SGBD**) es el programa que se ocupa de crear, guardar, proteger y consultar esos datos. Ejemplos de gestores son PostgreSQL, MySQL, SQLite y SQL Server.

Son dos cosas distintas, y mucha gente las confunde:

Concepto	Qué es	Ejemplo
Gestor (SGBD)	Un programa	PostgreSQL
Base de datos	Un conjunto de datos organizados	La base tienda con clientes y pedidos

¿Para qué sirve el gestor?

Es el **intermediario** entre las aplicaciones y los datos guardados en disco. Las aplicaciones no abren los archivos de datos directamente. Le piden las cosas al gestor y este las hace por ellas, aplicando las reglas y comprobando los permisos.

¿Por qué lo necesito?

Te permite distinguir dos frases que parecen iguales: «he instalado PostgreSQL» (el gestor) y «he creado la base de datos **tienda**» (los datos). Un mismo gestor puede contener muchas bases de datos independientes. Si no distingues ambas cosas, te resultará confuso leer documentación, pedir ayuda o configurar una conexión.

¿Cómo funciona?

Cuando una aplicación necesita un dato, ocurre este recorrido:

1. La aplicación envía una **petición** al gestor (una consulta).
2. El gestor comprueba que quien pregunta tiene **permiso**.
3. El gestor busca o modifica los datos en los **archivos** del disco.
4. El gestor devuelve el **resultado** a la aplicación.

La aplicación nunca toca los archivos. Eso es lo que garantiza que las reglas se cumplan siempre, venga la petición de la web, de la app del móvil o de una automatización.

Ejemplo sencillo

Piensa en una biblioteca. Los libros y las fichas son la **base de datos**. La bibliotecaria que busca los libros, los presta y los recoge es el **gestor**. Tú, que pides un libro, eres la **aplicación**. No entras al almacén a buscarlo tú: se lo pides a la bibliotecaria.

Ejemplo real

Un servidor tiene PostgreSQL instalado. Dentro de él hay tres bases de datos: **tienda**, **crm** y **pruebas**. El gestor es uno (PostgreSQL); las bases de datos son tres. Cada una tiene sus propias tablas, y una aplicación puede tener acceso a **tienda** y ninguno a **crm**.

Resultado visual

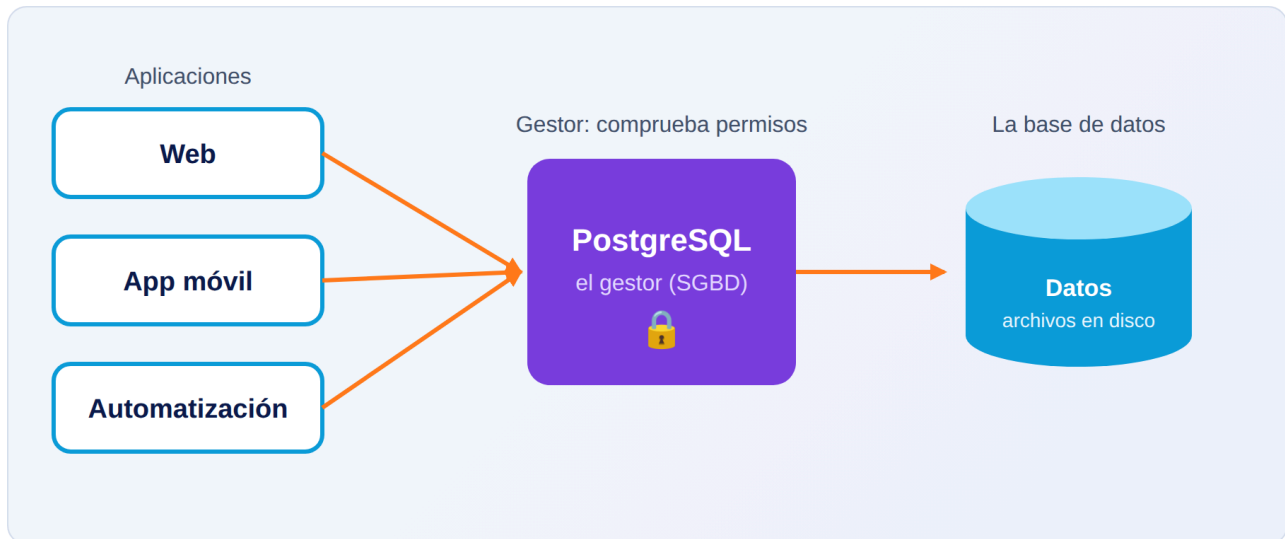


Figura 1.3-a. El gestor está entre las aplicaciones y los datos.

Qué ves en la imagen. A la izquierda están las tres aplicaciones que piden datos: una web, una app móvil y una automatización. En el centro está el gestor (PostgreSQL), con un candado que representa los permisos. A la derecha están los archivos de datos en el disco. Las flechas van siempre a través del gestor: ninguna aplicación llega directamente a los archivos.

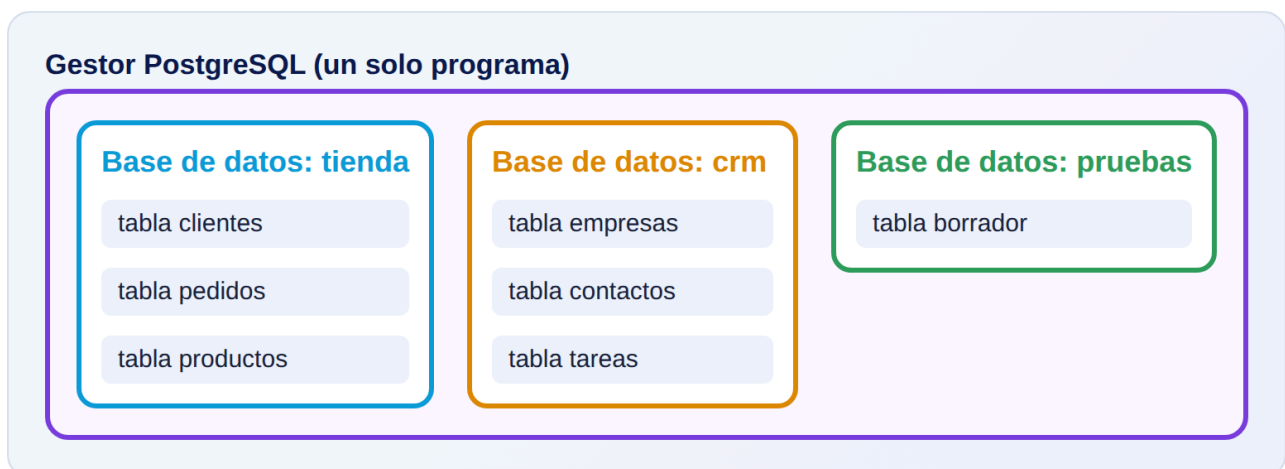


Figura 1.3-b. Un mismo gestor puede contener varias bases de datos.

Qué ves en la imagen. Dentro del bloque «Gestor PostgreSQL» hay tres bases de datos: **tienda**, **crm** y **pruebas**, y cada una tiene sus propias tablas. El gestor es único; las bases de datos son tres.

Ejercicio

1. Clasifica cada elemento como «gestor» o «base de datos»: PostgreSQL, la base `crm`, MySQL, la base `tienda`.
2. Completa la frase: «Para guardar mis clientes primero instalo ____ y después creo ____».
3. Una compañera te dice «mi base de datos PostgreSQL se ha caído». ¿Qué le preguntarías para saber qué ha fallado, el programa o los datos?

Solución

1. Gestor: PostgreSQL y MySQL. Base de datos: `crm` y `tienda`.
2. «Para guardar mis clientes primero instalo **un gestor (por ejemplo, PostgreSQL)** y después creo **una base de datos**».
3. Le preguntaría si lo que se ha caído es el servicio de PostgreSQL (el programa deja de responder) o si solo falla una base de datos concreta (por ejemplo, está corrupta o llena). Son problemas distintos con soluciones distintas.

Error habitual

Decir «base de datos PostgreSQL» para todo. PostgreSQL es el gestor; tus datos viven en bases de datos dentro de él. La confusión crea malentendidos al pedir ayuda o configurar herramientas.

Buena práctica

Usa cada palabra con su significado. Cuando hables o escribas, pregúntate: ¿me refiero al programa o a los datos?

Comprobación

Explica con tus palabras, sin usar ejemplos del texto, la diferencia entre gestor y base de datos. Después, di cuántas bases de datos puede tener un gestor.

1.4 Anatomía de una tabla

¿Qué es?

Una **tabla** es la forma de organizar los datos de un mismo tipo de cosa: clientes, pedidos, productos. Se parece a una cuadrícula de filas y columnas, como una hoja de cálculo, pero con reglas más estrictas.

Estas son sus partes:

- **Tabla:** el conjunto completo. Tiene un nombre, por ejemplo `clientes`.
- **Columna:** una característica que tienen todos los elementos. En `clientes` son `id`, `nombre`, `email` y `telefono`. Una columna también se llama **campo**: «campo» y «columna» son dos nombres para lo mismo.
- **Fila:** un elemento concreto de la tabla, por ejemplo un cliente. Una fila también se llama **registro**. Los dos nombres significan lo mismo.
- **Valor:** el dato que hay en el cruce de una fila y una columna, por ejemplo `luis@ejemplo.com`.
- **Cabecera:** la primera fila, que contiene los nombres de las columnas. No es un registro, solo rotula las columnas.

¿Para qué sirve?

Da una estructura fija. Todos los clientes tienen los mismos campos, así que se pueden buscar, ordenar, contar y comparar sin esfuerzo. Si cada cliente se guardase con campos distintos, no podrías preguntar «dame todos los emails».

¿Por qué lo necesito?

Todo lo que harás en este manual (diseñar, consultar, modificar, proteger) se apoya en tablas. Si no tienes claro qué es una fila y qué es una columna, el resto resultará confuso. Por eso conviene dedicar tiempo a este punto.

¿Cómo funciona?

Una tabla se define una vez: se decide su nombre y sus columnas, y qué tipo de dato admite cada una. Después se van añadiendo filas. Cada vez que llega un cliente nuevo, se añade una fila; cuando un cliente se da de baja, se elimina su fila. Las columnas no cambian con cada cliente: son la plantilla.

Una regla básica: **cada fila describe una sola cosa, y cada columna guarda una sola característica**. En una fila de `clientes` va un cliente, no dos; y el campo `email` guarda un email, no un email y un teléfono mezclados.

Ejemplo sencillo

La tabla CLIENTES con cuatro columnas y tres filas:

1. Se define: columnas `id` , `nombre` , `email` y `telefono` .
2. Llega Ana: se añade una fila con `1` , `Ana García` , `ana@ejemplo.com` y `600111222` .
3. Llega Luis: se añade otra fila. No sabemos su teléfono, así que esa celda queda sin valor (lo veremos en el punto 1.6).
4. Llega Marta: se añade la tercera fila.

Ejemplo real

Una tienda tiene la tabla `clientes` con 3.000 filas. Cuando abres tu cuenta en la web, la aplicación busca tu fila, lee tus valores (nombre, email, teléfono) y los muestra en pantalla. Cuando cambias tu teléfono, modifica un único valor en tu fila.

Resultado visual



id	nombre	email	telefono
1	Ana García	ana@ejemplo.com	600111222
2	Luis Pérez	luis@ejemplo.com	NULL
3	Marta Ruiz	marta@ejemplo.com	600333444

Fila = registro (Luis)
Columna = campo (email)
Valor

Figura 1.4-a. Tabla `CLIENTES`: la fila naranja es un registro; la columna azul es un campo.

Qué ves en la imagen. La tabla tiene la cabecera con los nombres de las columnas y tres filas. La fila de Luis está en naranja: es un **registro** completo. La columna `email` está en azul: es un **campo**. La celda en la que se cruzan, `luis@ejemplo.com` , es un **valor**.

Elemento	Qué representa en <code>CLIENTES</code>
Tabla	Todos los clientes del negocio
Columna (campo)	Una característica: <code>id</code> , <code>nombre</code> , <code>email</code> , <code>telefono</code>
Fila (registro)	Un cliente concreto, por ejemplo Luis
Valor	Un dato concreto, por ejemplo <code>luis@ejemplo.com</code>

Así se crea esta tabla en SQL. Es solo una muestra para ver la forma: se explica en la Parte 5.

```
CREATE TABLE clientes (
  id      integer PRIMARY KEY,
  nombre  text NOT NULL,
  email   text,
  telefono text
);
```

Lectura sencilla de este código: «crea una tabla llamada `clientes` con cuatro columnas». Cada línea entre paréntesis define una columna: su nombre y el tipo de dato que admite.

Ejercicio

1. En la figura 1.4-a, ¿cuántas filas y cuántas columnas hay? ¿Qué valor está en la fila de Marta, columna `email`?
2. Imagina una tabla `productos` para una tienda. Escribe cuatro columnas que tendría.
3. ¿Qué está mal en esta fila de `clientes`: `4 | Pedro Gil y Laura Sanz | pedro@ejemplo.com / laura@ejemplo.com | 600555666`?

Solución

1. Hay 3 filas (sin contar la cabecera) y 4 columnas. El valor de Marta en `email` es `marta@ejemplo.com`.
2. Por ejemplo: `id`, `nombre`, `precio_eur` y `stock`.
3. La fila mezcla dos personas (Pedro y Laura) y dos emails en una sola celda. Debería haber una fila por persona y un email por celda. Esta regla se estudia en profundidad en la normalización (Parte 4).

Error habitual

Confundir **tabla** con **base de datos**. Una base de datos contiene muchas tablas (`clientes`, `pedidos`, `productos`); una tabla guarda un solo tipo de cosa.

Buena práctica

Nombra las tablas en plural y en minúsculas (`clientes`) y las columnas en singular, también en minúsculas y sin espacios ni tildes (`nombre`, `fecha_alta`). Usa siempre el mismo estilo en todo el proyecto.

Comprobación

Dibuja una tabla con tres filas y tres columnas sobre un papel. Señala con un color una fila, con otro una columna y rodea un valor. Di cómo se llama cada cosa.

1.5 Tipos de datos

¿Qué es?

El **tipo de dato** indica qué clase de valor admite una columna. Los tipos básicos son:

- **Texto**: letras, números y símbolos tratados como texto (nombres, emails, descripciones).
- **Entero**: números sin decimales (cantidades, edades, contadores).
- **Decimal**: números con decimales (precios, medidas).
- **Fecha o fecha y hora**: un momento en el calendario.
- **Verdadero/falso** (también llamado **booleano**): solo admite dos valores, sí o no.

¿Para qué sirve?

Evita errores y habilita cálculos. Una columna de precios que solo admite números impide que alguien escriba «barato». Y como el gestor sabe que son números, puede sumarlos, compararlos y ordenarlos de forma correcta.

¿Por qué lo necesito?

Un ejemplo: si guardas los precios como texto, el gestor los ordena como si fueran palabras, y «100» queda antes que «20», porque compara letra a letra y «1» va antes que «2». Con el tipo correcto, 20 va antes que 100 y puedes calcular la suma de ventas del mes. El tipo no es un detalle: determina lo que podrás hacer con el dato.

¿Cómo funciona?

1. Al crear la tabla eliges un tipo para cada columna.
2. Cuando intentas guardar un valor, el gestor comprueba que encaja con el tipo.
3. Si no encaja, **rechaza** la operación y devuelve un error en lugar de guardar algo incorrecto.

Ejemplo sencillo

La tabla PRODUCTOS tiene estas columnas y tipos: `nombre` (texto), `stock` (entero), `precio_eur` (decimal), `alta` (fecha) y `activo` (verdadero/falso).

Ejemplo real

En una tienda, el `stock` es un entero porque no vendes media camiseta. El `precio_eur` es decimal porque hay céntimos. `alta` es una fecha para saber desde cuándo existe el producto, y `activo` es verdadero o falso para ocultarlo de la web sin borrarlo.

Resultado visual



nombre texto	stock entero	precio_eur decimal	alta fecha	activo verdadero/falso
Camiseta	12	19.95	2026-01-15	true
Gorra	0	9.90	2026-02-03	false
Mochila	5	34.50	2026-02-20	true

Figura 1.5-a. Tabla PRODUCTOS con el tipo de dato de cada columna.

Qué ves en la imagen. Encima de cada columna hay una etiqueta con su tipo. Debajo se ven las filas con valores que encajan: textos en `nombre`, enteros en `stock`, decimales en `precio_eur`, fechas en `alta` y verdadero/falso en `activo`.

Tipo (nombre general)	Para qué se usa	Ejemplo
Texto	Nombres, descripciones, emails	<code>Camiseta</code>
Entero	Cantidades y contadores	<code>12</code>
Decimal	Precios y medidas	<code>19.95</code>
Fecha / fecha y hora	Cuándo ocurrió algo	<code>2026-03-02</code>
Verdadero/falso	Sí o no	<code>true</code>

Cada gestor da nombres propios a sus tipos (por ejemplo, `text`, `integer`, `numeric`, `date` y `boolean` en PostgreSQL). Aprenderás los de PostgreSQL en la Parte 6; la idea es la misma en todos.

Ejercicio

1. Elige el tipo de dato para cada columna: la edad, el precio, el email, si el usuario está activo y la fecha de alta.
2. ¿Por qué un teléfono (por ejemplo, +34 600 111 222) conviene guardarlo como texto y no como entero?
3. Una columna `precio` guarda texto. Explica qué problema tendrás al intentar calcular el total de ventas.

Solución

1. Edad: entero. Precio: decimal. Email: texto. Activo: verdadero/falso. Fecha de alta: fecha.

2. Un teléfono no se suma ni se compara numéricamente, y puede llevar el signo «+», espacios o ceros iniciales que un entero perdería. Los identificadores que no se calculan se guardan como texto.
3. El gestor no puede sumar texto, o lo ordena letra a letra. Habría que convertir cada valor a número antes de calcular, y un valor mal escrito rompería el cálculo.

Error habitual

Guardarlo todo como texto «por si acaso». Funciona al principio, pero impide calcular, ordenar bien y validar. Elige el tipo según lo que vas a hacer con el dato.

Buena práctica

Para cada columna, pregúntate: ¿voy a sumarlo, compararlo, ordenarlo por fecha, o solo mostrarlo? La respuesta marca el tipo.

Comprobación

Escribe cinco columnas de cualquier tabla que se te ocurra y asigna un tipo a cada una. Para cada elección, di por qué ese tipo y no otro.

1.6 Valores, nulos e identificadores

¿Qué es?

Un **valor** es el contenido concreto de una celda. Los valores de una fila forman un registro.

NULL es un caso especial: significa «**no hay valor**». El dato no se conoce o no existe. Es importante entender que NULL **no es lo mismo** que:

- el número **0**, que es un valor conocido (cero);
- un **texto vacío** (`' '`), que es un valor conocido (un texto sin letras).

NULL es la ausencia de valor, no un valor.

Un **identificador** es un valor que distingue una fila de todas las demás. El más habitual es una columna `id`, un número que no se repite.

¿Para qué sirve?

NULL permite guardar una fila aunque falte algún dato opcional. El identificador permite señalar una fila concreta sin ambigüedad.

¿Por qué lo necesito?

Hay dos clientes llamados «Luis Pérez». Si alguien dice «cambia el teléfono de Luis Pérez», ¿de cuál habla? Con el identificador es inequívoco: «cambia el teléfono del cliente 2». Los nombres se repiten, los identificadores no.

Y si Luis no ha dado su teléfono, no puedes inventar uno. Tampoco quieres borrar a Luis por eso. Necesitas una forma de decir «el teléfono de Luis no se sabe»: eso es NULL.

¿Cómo funciona?

- Cada columna se define como **obligatoria** (no admite NULL) u **opcional** (admite NULL). Se decide al crear la tabla.
- El `id` es siempre obligatorio y único: el gestor rechaza una segunda fila con el mismo `id`.
- Cuando un cálculo toca un NULL, el resultado suele ser «desconocido». Por eso conviene tratar los NULL con cuidado (lo veremos en la Parte 5, con `IS NULL` y `COALESCE`).

Ejemplo sencillo

Luis no ha dado su teléfono. En su fila, la celda `telefono` es NULL. Su fila sigue siendo válida: tiene `id`, `nombre` y `email`.

Ejemplo real

Una tabla `suscripciones` tiene una columna `fecha_baja`. Mientras el cliente sigue activo, `fecha_baja` es NULL. Cuando se da de baja, se rellena con la fecha. Para saber quién está activo basta con buscar las filas donde `fecha_baja` no tiene valor.

Resultado visual

tienda · clientes

id	nombre	email	telefono
1	Ana García	ana@ejemplo.com	600111222
2	Luis Pérez	luis@ejemplo.com	NULL
3	Marta Ruiz	marta@ejemplo.com	600333444

Tres casos que parecen iguales

NULL — no se sabe / no hay valor

0 — valor conocido: cero

" — valor conocido: texto sin letras

Figura 1.6-a. Luis tiene el teléfono en NULL: falta el dato.

Qué ves en la imagen. En la fila de Luis, la celda `telefono` muestra `NULL` en gris cursiva: no hay valor. A la derecha se comparan tres casos que parecen iguales y no lo son: `NULL` (no se sabe), `0` (valor conocido: cero) y texto vacío (valor conocido: un texto sin letras).

Ejercicio

1. Un producto no tiene descuento. ¿Pondrías 0, un texto vacío o NULL? Razona la respuesta.
2. Decide si cada columna de `clientes` debería ser obligatoria u opcional: `id`, `nombre`, `email`, `telefono`.
3. Dos clientes se llaman «Luis Pérez». ¿Cómo distingue la base de datos a uno de otro?

Solución

1. Depende del significado. Si «sin descuento» equivale a un 0 %, usa 0, porque es un valor conocido. Si el descuento aún no se ha decidido, usa NULL. Lo importante es decidirlo una vez y ser coherente.
2. `id`: obligatoria (identifica la fila). `nombre`: obligatoria (sin nombre la fila no tiene sentido). `email` y `telefono`: opcionales, salvo que tu negocio exija contactar siempre por uno de ellos.
3. Por su `id`. Cada Luis Pérez tiene un `id` distinto, aunque el nombre sea el mismo.

Error habitual

Usar valores inventados como «-1», «sin dato» o «N/A» en lugar de NULL. Esos valores falsos se cuelan en los cálculos (una media que incluye «-1») y confunden a quien lea los datos después.

Buena práctica

Haz obligatorias (NOT NULL) las columnas sin las cuales la fila no tiene sentido, como el nombre. Usa NULL solo cuando la ausencia del dato sea legítima.

Comprobación

Elige una tabla de tu proyecto. Para cada columna escribe «obligatoria» u «opcional». Después, para cada opcional, escribe qué significa que esté vacía.

1.7 Claves: primaria y extranjera

¿Qué es?

Una **clave** es una columna (o un grupo de columnas) que sirve para identificar o enlazar filas. Hay dos tipos esenciales:

- La **clave primaria** (en inglés, *primary key*, abreviada **PK**) es la columna que identifica de forma única cada fila de una tabla. En CLIENTES es el `id`. Nunca se repite y nunca está vacía.
- La **clave extranjera** (en inglés, *foreign key*, abreviada **FK**) es una columna que guarda la clave primaria de **otra** tabla, para enlazar las dos. En PEDIDOS, la columna `cliente_id` guarda el `id` del cliente que hizo el pedido.

¿Para qué sirve?

La clave primaria permite señalar una fila concreta sin duda posible. La clave extranjera permite **relacionar tablas sin repetir datos**: en lugar de copiar el nombre y el teléfono de Ana en cada pedido, el pedido solo guarda el número 1, que es el `id` de Ana. Para ver los datos de Ana, se va a la tabla de clientes y se busca ese número.

¿Por qué lo necesito?

Sin claves no podrías enlazar un pedido con su cliente, ni garantizar que no haya dos filas idénticas. Y si copiaras los datos del cliente en cada pedido, volverías al problema del punto 1.2: datos repetidos que se contradicen.

¿Cómo funciona?

El gestor vigila las claves por ti:

1. **Con la clave primaria**: rechaza cualquier fila nueva cuyo `id` ya exista.
2. **Con la clave extranjera**: rechaza un pedido cuyo `cliente_id` no corresponda a ningún cliente que exista. No puede haber un pedido «de nadie».
3. **Al borrar**: puede impedir que borres un cliente que aún tiene pedidos, para no dejar pedidos huérfanos (esto se configura; se estudia en la Parte 3).

Ejemplo sencillo

El pedido 101 tiene `cliente_id` = 1. Buscas el 1 en la tabla de clientes: es Ana García. El pedido 102 tiene `cliente_id` = 3: es Marta Ruiz.

Ejemplo real

En una tienda con 20.000 pedidos, la tabla `pedidos` solo guarda un número por pedido para identificar al cliente. Cuando Ana cambia su teléfono, se modifica una sola fila en `clientes` y todos sus pedidos «ven» el dato nuevo automáticamente, porque nunca lo copiaron.

Resultado visual

tienda · clientes		
id PK	nombre	email
1	Ana García	ana@ejemplo.com
2	Luis Pérez	luis@ejemplo.com
3	Marta Ruiz	marta@ejemplo.com

cliente_id apunta al id

tienda · pedidos				
id PK	cliente_id FK	fecha	total	
101	1	2026-03-02	45.90	
102	3	2026-03-05	12.00	
103	1	2026-03-09	80.50	

☐ Ana (pedidos 101 y 103)
 ☒ Marta (pedido 102)
 ☐ Luis (sin pedidos)

Figura 1.7-a. Claves primaria (PK) y extranjera (FK) enlazando PEDIDOS con CLIENTES.

Qué ves en la imagen. A la izquierda, la tabla CLIENTES con su clave primaria `id` marcada con PK. A la derecha, la tabla PEDIDOS con su propia clave primaria (`id`) y la clave extranjera `cliente_id`, marcada con FK. Cada pedido tiene el mismo color que su cliente: Ana (azul) tiene los pedidos 101 y 103, Marta (naranja) el 102 y Luis, sin pedidos, no aparece en ellos. El enlace es el número que hay en `cliente_id`.

Así se declara la relación en SQL. De nuevo, es una muestra para ver la forma.

```
CREATE TABLE pedidos (
  id          integer PRIMARY KEY,
  cliente_id integer REFERENCES clientes (id),
  fecha       date,
  total       numeric(10, 2)
);
```

Lectura sencilla: «la columna `cliente_id` solo puede contener un número que exista en la columna `id` de `clientes`».

```
INSERT INTO pedidos (id, cliente_id, fecha, total)
VALUES (104, 9, '2026-03-12', 20.00);
```

```
ERROR: insert or update on table "pedidos" violates foreign key constraint
"pedidos_cliente_id_fkey"
DETAIL: Key (cliente_id)=(9) is not present in table "clientes".
```

Resultado: el pedido no se guarda. No existe el cliente 9.

Ilustración: el texto exacto depende del gestor.

Figura 1.7-b. Ilustración: el gestor rechaza un pedido de un cliente que no existe.

Qué ves en la imagen. Se intenta guardar un pedido con `cliente_id` = 9, pero no hay ningún cliente con ese `id`. El gestor rechaza la operación y muestra un error. El pedido no se guarda: la relación queda protegida. El texto exacto del mensaje depende del gestor; veremos uno real en la Parte 6.

Ejercicio

1. En la figura 1.7-a, ¿de quién es el pedido 102? ¿Cuántos pedidos tiene Ana?
2. Luis quiere hacer un pedido nuevo. ¿Qué valor tendría `cliente_id`?
3. ¿Qué pasaría, según lo explicado, si intentas guardar un cliente con `id` = 2, que ya existe?

Solución

1. El pedido 102 es de Marta (`cliente_id` = 3). Ana tiene 2 pedidos: el 101 y el 103.
2. 2, que es el `id` de Luis. Su nombre no se escribe en el pedido: solo su `id`.
3. El gestor lo rechazaría, porque la clave primaria no admite repeticiones. Habría que usar un `id` nuevo, por ejemplo el 4.

Error habitual

Usar el nombre o el email como clave primaria. Pueden cambiar (alguien se casa, cambia de correo) o repetirse (dos «Luis Pérez»), y si cambian hay que actualizarlos en todas las tablas que los usan. Un `id` sin significado propio nunca tiene que cambiar.

Buena práctica

Dale a cada tabla una clave primaria, y usa claves extranjeras para toda relación entre tablas. Así el gestor protege la coherencia de tus datos sin que tengas que vigilarla a mano.

Comprobación

Para cada tabla de tu diseño, responde: ¿qué columna identifica cada fila? ¿Con qué otras tablas se enlaza y mediante qué columna?

1.8 Índices

¿Qué es?

Un **índice** es una estructura que la base de datos mantiene aparte de la tabla para encontrar filas rápidamente. Funciona igual que el índice alfabético de un libro: en lugar de leer las 400 páginas para encontrar la palabra «bosque», vas al índice, ves en qué página está y saltas directamente a ella.

¿Para qué sirve?

Acelera las búsquedas por una columna, por ejemplo buscar un cliente por su email o los pedidos de un cliente por `cliente_id`.

¿Por qué lo necesito?

Con 10 filas, da igual cómo se busque. Con 5 millones, recorrer la tabla fila a fila para encontrar un email puede ser lento, y esa lentitud se repite cada vez que alguien inicia sesión. Con un índice, la búsqueda es casi inmediata. En proyectos grandes, los índices son una de las herramientas que más rendimiento ganan.

¿Cómo funciona?

Sin índice, el gestor hace una **lectura completa** de la tabla: mira fila por fila hasta encontrar la que busca. Con índice:

1. El gestor guarda aparte los valores de la columna **ordenados**, cada uno con la posición de su fila.
2. Cuando se busca un valor, lo localiza en esa lista ordenada (más rápido, porque al estar ordenada no hay que mirarla toda).
3. Salta directamente a la fila indicada.

Esta es una simplificación: en las Partes 6 y 18 veremos los tipos de índice y cómo comprobar si se están usando.

Los índices tienen un coste: ocupan espacio y hay que actualizarlos cada vez que se inserta, modifica o borra una fila. Por eso no se crean en todas las columnas.

Ejemplo sencillo

Buscas «marta@ejemplo.com». Sin índice: el gestor mira la fila 1 (no es), la 2 (no es) y la 3 (sí). Con índice: ve en la lista ordenada que ese email está en la fila 3 y va directo.

Ejemplo real

La tabla `pedidos` tiene 5 millones de filas. La web muestra «Mis pedidos» y busca los de un cliente por `cliente_id`. Con un índice sobre `cliente_id`, el gestor localiza esos pedidos sin recorrer los 5 millones. El efecto real se mide en la Parte 18 con EXPLAIN.

Resultado visual

Índice de un libro		Índice de la columna email	
Palabra	Página	email (ordenado)	fila
bosque	212	ana@ejemplo.com	1
cielo	45	luis@ejemplo.com	2
río	133	marta@ejemplo.com	3
sol	301		

Ordenado: vas directo a la página

Buscas marta@... → fila 3, sin recorrer todo

Figura 1.8-a. El índice de un libro y el índice de una columna funcionan igual.

Qué ves en la imagen. A la izquierda, el índice de un libro: una lista ordenada de palabras, cada una con su página. A la derecha, el índice de la columna `email` de `CLIENTES`: una lista ordenada de emails, cada uno con el número de fila donde está. Ambos sirven para saltar directamente al sitio sin leer todo.

```
CREATE INDEX idx_clientes_email ON clientes (email);
```

Lectura sencilla: «crea un índice llamado `idx_clientes_email` sobre la columna `email` de la tabla `clientes`».

Ejercicio

1. ¿Qué columna de `CLIENTES` indexarías si tu aplicación busca clientes por correo?
2. ¿Qué columna de `PEDIDOS` indexarías para mostrar «los pedidos de este cliente»?
3. ¿Por qué no es buena idea crear un índice en cada columna de todas las tablas?

Solución

1. La columna `email`.
2. La columna `cliente_id`, que es por la que se busca.
3. Cada índice ocupa espacio y hay que actualizarlo en cada inserción, modificación o borrado. Demasiados índices ralentizan las escrituras y consumen disco sin aportar nada si nadie busca por esas columnas.

Error habitual

Poner índices a todo «por si acaso», o pensar que un índice sirve para guardar más datos o para ordenar la tabla visualmente. Un índice solo ayuda a **encontrar**.

Buena práctica

Indexa las columnas por las que buscas, enlazas u ordenas con frecuencia, y mide antes y después. Una columna que casi nunca se consulta no necesita índice.

Comprobación

Explica con tus palabras por qué un índice acelera las búsquedas y por qué tiene un coste. Después, nombra una columna de tu proyecto que sí indexarías y otra que no.

1.9 Consultas

¿Qué es?

Una **consulta** es una petición que se le hace a la base de datos. Cuando la petición es una pregunta («dame los clientes»), la base de datos responde con filas. Las consultas se escriben en **SQL** (se pronuncia «ese-cu-ele»), el lenguaje de las bases de datos relacionales. Aprenderás SQL desde cero en la Parte 5.

Una consulta puede pedir datos, pero también crearlos, modificarlos o borrarlos. En el uso diario, casi todas las consultas son preguntas.

¿Para qué sirve?

Para sacar información de los datos guardados: listar, buscar, filtrar, ordenar, contar, sumar. Y para cambiar los datos de forma controlada.

¿Por qué lo necesito?

Guardar datos no sirve de nada si no puedes recuperarlos. Cada pantalla de una aplicación (la lista de pedidos, el perfil, el buscador) ejecuta una o varias consultas por detrás. Entender qué es una consulta te permite entender cómo funciona cualquier aplicación.

¿Cómo funciona?

1. Escribes (o la aplicación escribe) la consulta.
2. La envías al gestor.
3. El gestor la ejecuta y construye una **tabla de resultados**. Puede tener cero, una o muchas filas.
4. El resultado se devuelve a quien preguntó.

El resultado no es una tabla guardada: es una tabla temporal, creada para esa pregunta.

Ejemplo sencillo

«Dame el nombre y el email de todos los clientes». En SQL (muestra para ver la forma):

```
SELECT nombre, email
FROM clientes;
```

Lectura sencilla: «selecciona **nombre** y **email** de la tabla **clientes** ». Devuelve tres filas.

Ejemplo real

Al abrir «Mis pedidos» en una tienda, la aplicación pregunta a la base de datos por los pedidos de tu usuario, y con las filas devueltas dibuja la lista en pantalla.

Resultado visual

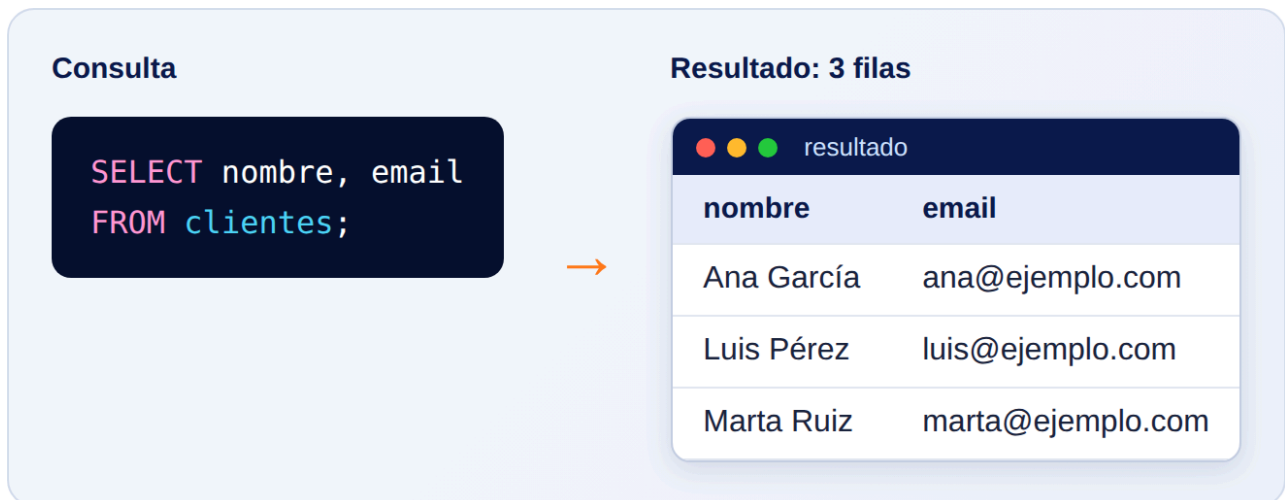


Figura 1.9-a. Una consulta y su resultado.

Qué ves en la imagen. A la izquierda está la consulta; a la derecha, la tabla de resultados que devuelve: solo las columnas `nombre` y `email`, con las tres filas de `CLIENTES`. Las columnas `id` y `telefono` no aparecen porque no se pidieron.

Una consulta también puede filtrar. Esta pide solo los pedidos de más de 40 euros:

```
SELECT id, cliente_id, total
FROM pedidos
WHERE total > 40;
```

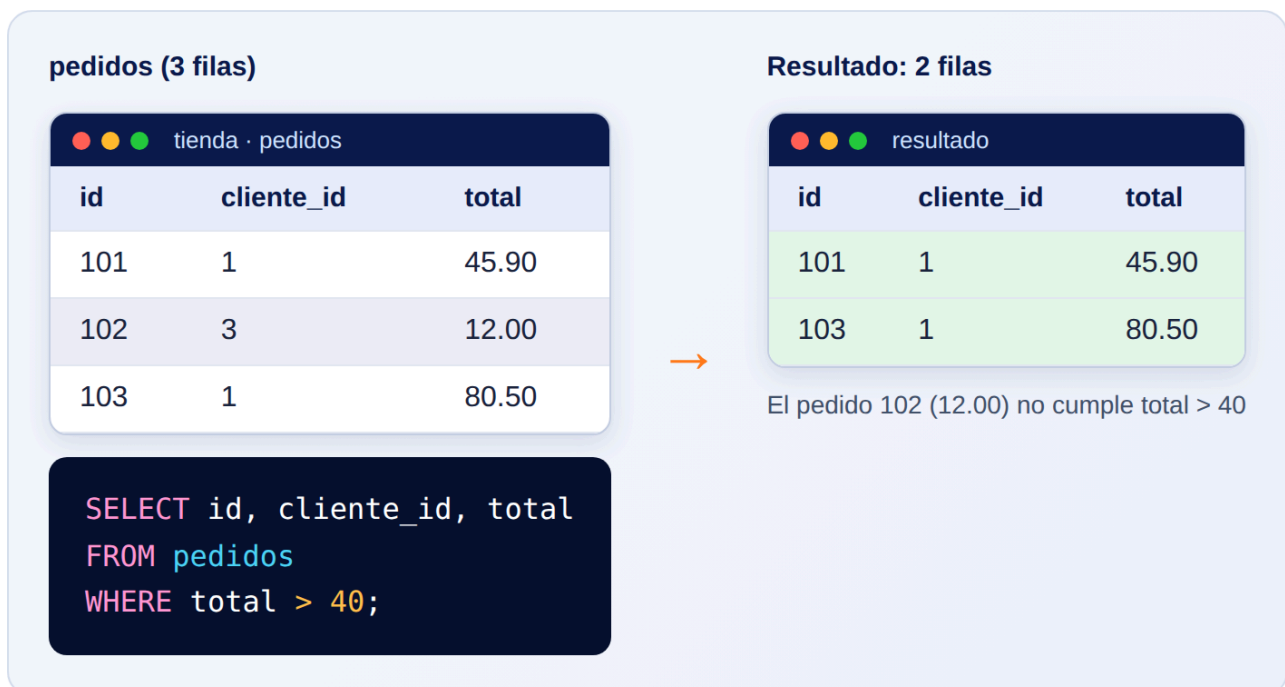


Figura 1.9-b. Una consulta con filtro: solo salen las filas que cumplen la condición.

Qué ves en la imagen. A la izquierda están los tres pedidos completos; a la derecha, el resultado. El pedido 102, de 12,00 euros, no cumple la condición y no aparece. Salen el 101 (45,90) y el 103

(80,50).

Ejercicio

1. Con los datos de la figura 1.7-a, formula en español la pregunta «pedidos de más de 40 euros» y di cuántas filas devolvería.
2. Escribe en español qué pedirías para obtener solo los pedidos de Ana.
3. ¿Por qué conviene pedir solo las columnas necesarias en lugar de todas?

Solución

1. Devolvería 2 filas: el pedido 101 (45,90) y el 103 (80,50). El 102 (12,00) no llega a 40.
2. «Dame los pedidos cuyo `cliente_id` sea 1» (el `id` de Ana). Devolvería los pedidos 101 y 103.
3. Pedir menos datos hace la consulta más rápida y la respuesta más pequeña, y evita exponer información que no se necesita (por ejemplo, datos personales).

Error habitual

Pedir siempre todos los datos de todas las filas «por si acaso». Con tablas grandes, eso consume tiempo, memoria y red sin necesidad.

Buena práctica

Formula primero la pregunta en español; después, la consulta. Y antes de ejecutarla, calcula cuántas filas esperas obtener: así detectarás los errores de lógica.

Comprobación

Antes de ejecutar una consulta, ¿sabrías decir qué columnas devolverá y cuántas filas aproximadamente? Si no, aún no has formulado bien la pregunta.

1.10 El entorno: cliente, servidor, API, VPS y nube

¿Qué es?

En este manual aparecerán cinco palabras del entorno técnico. Las explicamos desde cero:

Término	Qué significa
Cliente	El programa o dispositivo que pide cosas: tu navegador, la app del móvil, una automatización de n8n
Servidor	Un ordenador (o un programa) que está siempre encendido y responde a las peticiones de otros
API	Una «ventanilla» por la que un programa pide o envía datos a otro, con reglas fijas sobre qué se puede pedir y cómo
VPS	Un servidor virtual alquilado: una parte de un ordenador de un proveedor que tú controlas como si fuera tuyo
Base de datos en la nube	Una base de datos que alquilas ya instalada, configurada y mantenida por un proveedor

Para entender «servidor», piensa en un restaurante. El cliente es el comensal que pide. El servidor es la cocina, que siempre está disponible y prepara lo que se le pide. La API es la carta y el camarero: dicen qué se puede pedir y cómo se pide.

¿Para qué sirve?

Saber dónde vive cada pieza te permite entender cómo viaja un dato desde la pantalla hasta el disco, y dónde puede fallar. Cuando algo no funciona, lo primero que se hace es preguntar: ¿en qué pieza está el problema?

¿Por qué lo necesito?

Tu aplicación y tu base de datos casi nunca están en el mismo sitio. La web se ejecuta en el navegador de cada usuario; la base de datos, en otro ordenador. Entre ambas hay piezas intermedias. Entender este recorrido es la base para desplegar, proteger y diagnosticar tus proyectos.

¿Cómo funciona?

El recorrido típico de un dato es este:

1. El **cliente** (navegador o móvil) envía una petición a la **API**.
2. La API, que se ejecuta en un **servidor**, comprueba quién eres y qué puedes ver.
3. La API pregunta a la **base de datos**.
4. La base de datos responde a la API.

5. La API devuelve la respuesta al cliente, que la muestra en pantalla.

Normalmente el cliente **no** habla directamente con la base de datos: una API está en medio. Se explica por qué en la Parte 15.

Dónde vive el servidor puede variar:

- En un **VPS** que tú alquilas y administras: tienes control total, y también la responsabilidad de mantenerlo (Parte 8).
- En un servicio de **base de datos en la nube**, donde el proveedor se encarga de instalarlo y mantenerlo (Partes 9 y 10).

Ejemplo sencillo

Abres una app y pulsas «Mis pedidos». Tu móvil (cliente) pide los pedidos a la API. La API pregunta a la base de datos. La base de datos devuelve las filas, la API las envía a tu móvil y tu móvil las dibuja.

Ejemplo real

Una pequeña empresa alquila un VPS, instala en él PostgreSQL y su API, y los usa desde una web y una app móvil. Otra empresa alquila solo la base de datos a un proveedor y pone la API en otro servicio. En ambos casos el recorrido de una petición es el mismo; cambia dónde vive cada pieza.

Resultado visual

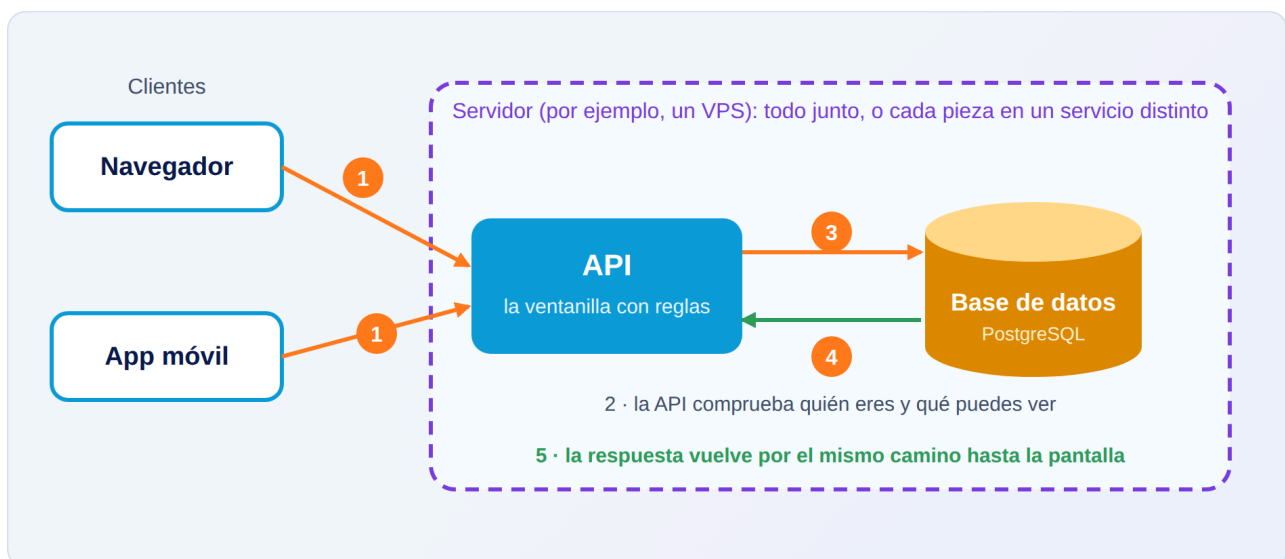


Figura 1.10-a. Recorrido de una petición: cliente, API, servidor y base de datos.

Qué ves en la imagen. A la izquierda están los clientes (navegador y móvil). Las flechas con números muestran el recorrido de la petición hacia la API (en un servidor) y hacia la base de datos, y la respuesta que vuelve por el mismo camino. El recuadro punteado indica que el servidor con la API y la base de datos pueden vivir en un mismo VPS o en servicios distintos.

Ejercicio

1. Ordena el recorrido de una petición: base de datos, navegador, API.
2. Clasifica como cliente, servidor o API: la app de tu móvil, el ordenador de un proveedor que está siempre encendido, la «ventanilla» por la que se piden los pedidos.
3. Explica con tus palabras la diferencia entre alquilar un VPS e instalar tú PostgreSQL, y alquilar una base de datos en la nube.

Solución

1. Navegador → API → base de datos. La respuesta vuelve por el mismo camino.
2. La app del móvil: cliente. El ordenador del proveedor: servidor. La ventanilla: API.
3. En un VPS tú instalas, configuras, actualizas y proteges PostgreSQL: control total y más trabajo. En una base de datos en la nube el proveedor se ocupa de buena parte del mantenimiento, a cambio de menos control y de pagar por el servicio. Lo comparamos con detalle en la Parte 9.

Error habitual

Pensar que el navegador o la app hablan directamente con la base de datos. Si fuera así, las contraseñas de la base de datos estarían dentro de la aplicación, al alcance de cualquiera que la inspeccione. Por eso se usa una API en medio.

Buena práctica

Antes de montar cualquier proyecto, dibuja en un papel el recorrido de los datos: quién pide, quién responde, dónde está cada pieza.

Comprobación

Para uno de tus proyectos, escribe en una línea dónde vive cada pieza: cliente, API, base de datos. Si alguna no la sabes situar, es la primera que debes aclarar.

Resumen de la Parte 1

- Los datos sueltos no dicen nada; con contexto se convierten en información.
- Las bases de datos existen para guardar muchos datos sin duplicados, contradicciones ni pérdidas, y para permitir el uso simultáneo.
- El gestor es el programa; la base de datos son los datos. Un gestor contiene muchas bases de datos.
- Una tabla tiene columnas (campos) y filas (registros); cada celda contiene un valor.
- Cada columna tiene un tipo de dato; NULL significa «no hay valor».
- La clave primaria identifica filas; la clave extranjera enlaza tablas.
- Los índices aceleran las búsquedas, pero tienen un coste.

- Una consulta es una petición al gestor y devuelve una tabla de resultados.
- Normalmente, el cliente pide a una API, y la API consulta la base de datos.

Glosario de la Parte 1

Término	Significado
Dato	Valor suelto, sin contexto
Información	Datos con contexto que les da sentido
Base de datos	Conjunto organizado de datos relacionados
SGBD (gestor)	Programa que crea, guarda, protege y consulta bases de datos
Tabla	Datos de un mismo tipo de cosa, en filas y columnas
Columna / campo	Una característica que tienen todos los elementos de la tabla
Fila / registro	Un elemento concreto de la tabla
Valor	Dato que hay en el cruce de una fila y una columna
Tipo de dato	Clase de valor que admite una columna
NULL	Ausencia de valor
Clave primaria (PK)	Columna que identifica cada fila de forma única
Clave extranjera (FK)	Columna que guarda la clave primaria de otra tabla
Índice	Estructura auxiliar que acelera las búsquedas
Consulta	Petición que se hace al gestor
SQL	Lenguaje con el que se escriben las consultas
Cliente	Quien pide datos (navegador, app, automatización)
Servidor	Ordenador o programa que responde a peticiones
API	Ventanilla con reglas por la que un programa pide datos a otro
VPS	Servidor virtual alquilado

Mini examen de la Parte 1

Responde sin mirar las respuestas; después compáralas.

1. ¿Qué diferencia hay entre un dato y una información? Pon un ejemplo.
2. ¿Qué diferencia hay entre PostgreSQL y una base de datos llamada **tienda** ?

3. En la tabla CLIENTES, ¿qué es una fila y qué es una columna?
4. ¿Qué tipo de dato usarías para un precio y por qué?
5. ¿Qué significa NULL y en qué se diferencia de 0?
6. ¿Para qué sirve la clave primaria?
7. En PEDIDOS, ¿qué columna es la clave extranjera y a qué apunta?
8. ¿Qué ventaja tiene un índice y qué coste?
9. ¿Qué devuelve una consulta?
10. Ordena el recorrido: base de datos, API, móvil.

Respuestas

1. Un dato es un valor sin contexto («34»); la información tiene contexto («Ana tiene 34 años»).
2. PostgreSQL es el gestor (el programa); `tienda` es una base de datos que vive dentro de él.
3. Una fila es un registro (un cliente concreto); una columna es un campo (una característica, como `email`).
4. Decimal, porque necesita céntimos y se va a sumar y comparar.
5. NULL significa «no hay valor»; 0 es un valor conocido (cero).
6. Para identificar cada fila de forma única, sin repeticiones ni vacíos.
7. `cliente_id`, que apunta al `id` de CLIENTES.
8. Acelera las búsquedas; cuesta espacio y tiempo al escribir, porque hay que mantenerlo actualizado.
9. Una tabla de resultados con cero, una o muchas filas.
10. Móvil → API → base de datos (y la respuesta vuelve por el mismo camino).

Si has acertado 8 o más, estás listo para la Parte 2. Si no, repasa los puntos de las preguntas falladas.